SD242 – Verificação de Sistemas Digitais (80h)

Autores	
Anderson Max Almeida e Porfirio (Hbr)	Nome 2 (INSTITUIÇÃO)
Nome 3 (INSTITUIÇÃO)	Nome 4 (INSTITUIÇÃO)

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

Tópicos

- Introdução
- UVM
- Componentes Principais
- UVM Components
- Standard UVM Phases
- Build Phases
- Run Time Phases
- Parametrização

Página em branco.

Aula 16
Criação e Configuração de Testbenchs

Tópicos

- UVM Configuration Database
- Assinatura dos métodos
- Get()
- Set()
- Virtual Interface
- Configuração do Agent
- Configuração do Environment
- Configuração do Test
- Precedência
- Desenvolvimento de um ambiente completo

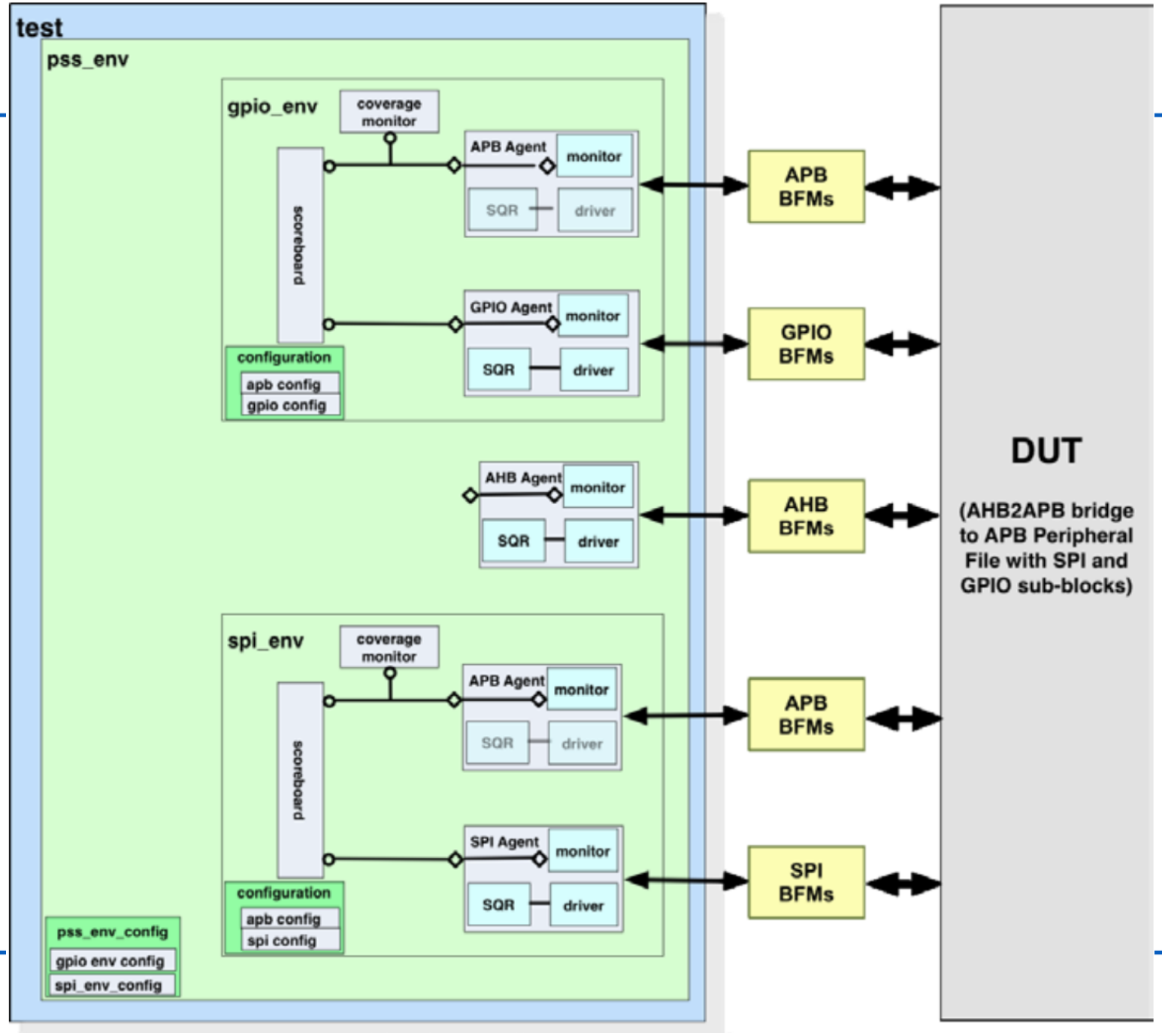
UVM Configuration Database

- uvm_config_db
  - Qualquer informação que compartilhada entre dois ou mais componentes ou objetos em um ambiente UVM, deve usar a classe uvm_config_db.
  - Esse recurso passa interfaces virtuais do domínio HDL/DUT para o teste.
  - E passa objetos de configuração para baixo através da hierarquia do testbench.

  - uvm_config_db::**set** para colocar informação no banco de dados.
  - uvm_config_db::**get** para recuperar informação do banco de dados.

  - A classe é parametrizada por tipo, onde o banco de dados se comporta como se estivesse particionado em "mini bancos de dados" específicos por tipo.

Env_config


test
pss_env
gpio_env
coverage monitor
APB Agent
monitor
SQR
driver
scoreboard
GPIO Agent
monitor
SQR
driver
configuration
apb config
gpio config
AHB Agent
monitor
SQR
driver
spi_env
coverage monitor
APB Agent
monitor
SQR
driver
scoreboard
SPI Agent
monitor
SQR
driver
configuration
apb config
spi config
pss_env_config
gpio env config
spi_env_config
APB BFMs
GPIO BFMs
AHB BFMs
APB BFMs
SPI BFMs
DUT
(AHB2APB bridge to APB Peripheral File with SPI and GPIO sub-blocks)

Assinatura dos métodos

- uvm_config_db
  - A assinatura completa dos métodos/funções
    - void uvm_config_db #(type T = int)::**set**(uvm_component cntxt , string inst_name , string field_name , T value);
    - bit uvm_config_db #(type T = int)::**get**(uvm_component cntxt , string inst_name , string field_name , ref T value);
    - **T** é o tipo do recurso, ou elemento - geralmente uma interface virtual ou um objeto de configuração.
    - **cntxt** e **inst_name** formam um escopo que é usado para localizar o recurso dentro do banco de dados; {cntxt.get_full_name(),".",inst_name}.
    - field_name é o nome dado ao recurso.
    - value é o valor real ou referência que é colocado no banco de dados.
    - Get() retorna 1 se dado recuperado ou 0 indicando que nenhum recurso desse tipo e com esse contexto e nome existe no banco de dados.

UVM Configuration Database

- SET()
  - interface ahb_if data_port_if(clk , reset); // no modulo top_tb
  - interface ahb_if control_port_if(clk , reset);
  - ...
  - uvm_config_db #(virtual ahb_if)::set(null , "uvm_test_top" , "data_port" , data_port_if);
  - uvm_config_db #(virtual ahb_if)::set(null , "uvm_test_top" , "control_port" , control_port_if);

  - class env extends uvm_env;
  - ahb_agent_config m_ahb_agent_config;
  - function void build_phase(uvm_phase phase);
  - ...
  - m_ahb_agent = ahb_agent::type_id::create("m_ahb_agent" , this);
  - uvm_config_db #(ahb_agent_config)::set(this , "m_ahb_agent*" ,"ahb_agent_config" , m_ahb_agent_config); ...
  - endfunction
  - endclass

UVM Configuration Database

- GET()

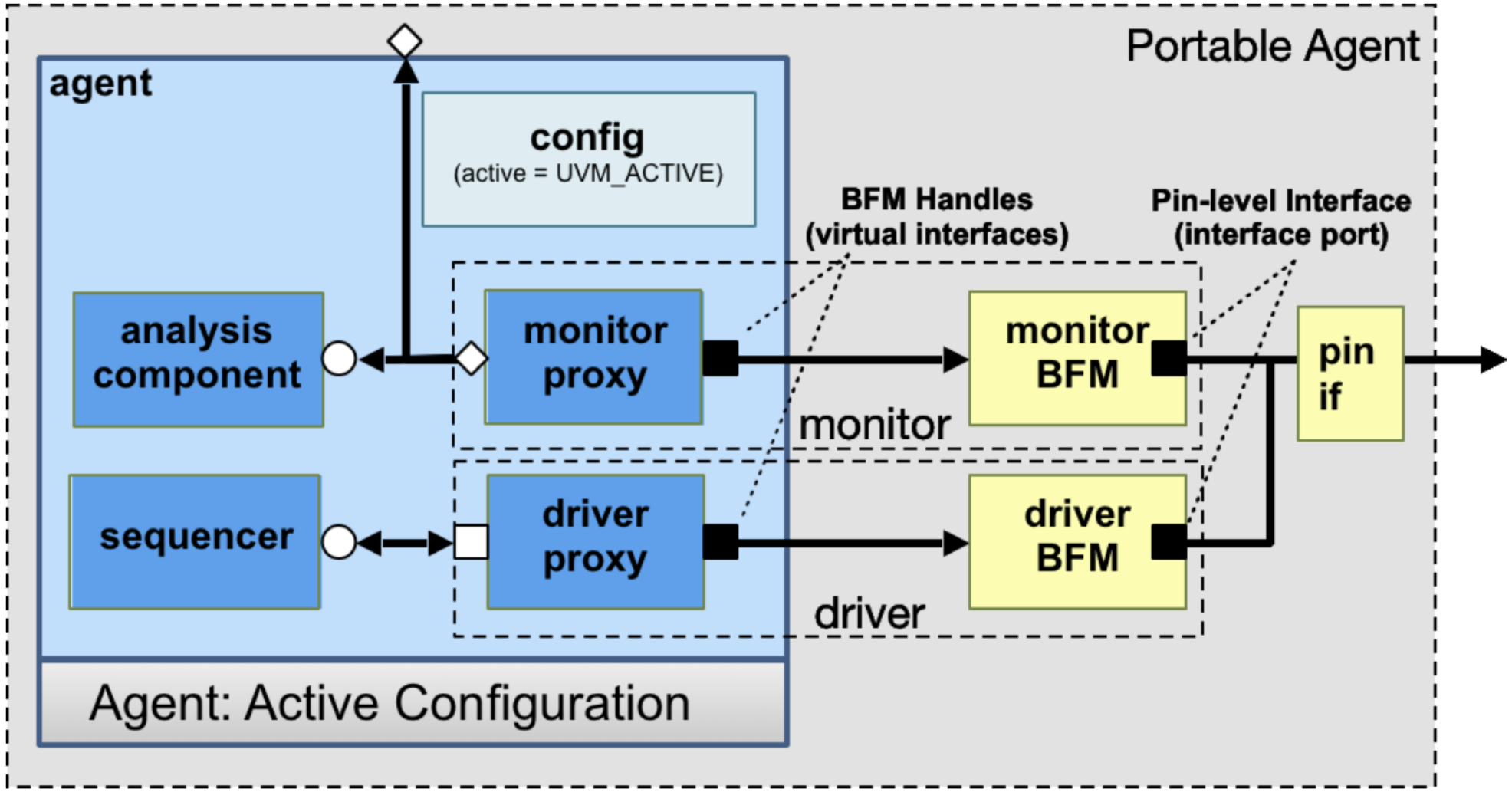
```
class test extends uvm_test;
  ...
  function void build_phase( uvm_phase phase );
      if(!uvm_config_db #( virtual ahb_if )::get( this , "" , "data_port",
          m_cfg.m_data_port_config.m_ahb_if)) begin
          `uvm_error("Config Error", "uvm_config_db #( virtual ahb_if )::get cannot find data_port" ))
       end
    ...
    endfunction
  ...
  endclass
```

UVM Configuration Database

- GET()

```
class ahb_monitor extends uvm_monitor;
    ahb_agent_config m_cfg;
    function void build_phase( uvm_phase phase );
        if( !uvm_config_db #( ahb_agent_config )::get( this , "" , "ahb_agent_config" , m_cfg ) ) begin
            `uvm_error("Config Error" , "uvm_config_db #( ahb_agent_config)::get cannot find
            ahb_agent_config" )
        end
    endfunction
endclass
```

Virtual Interface - uvm_config_db



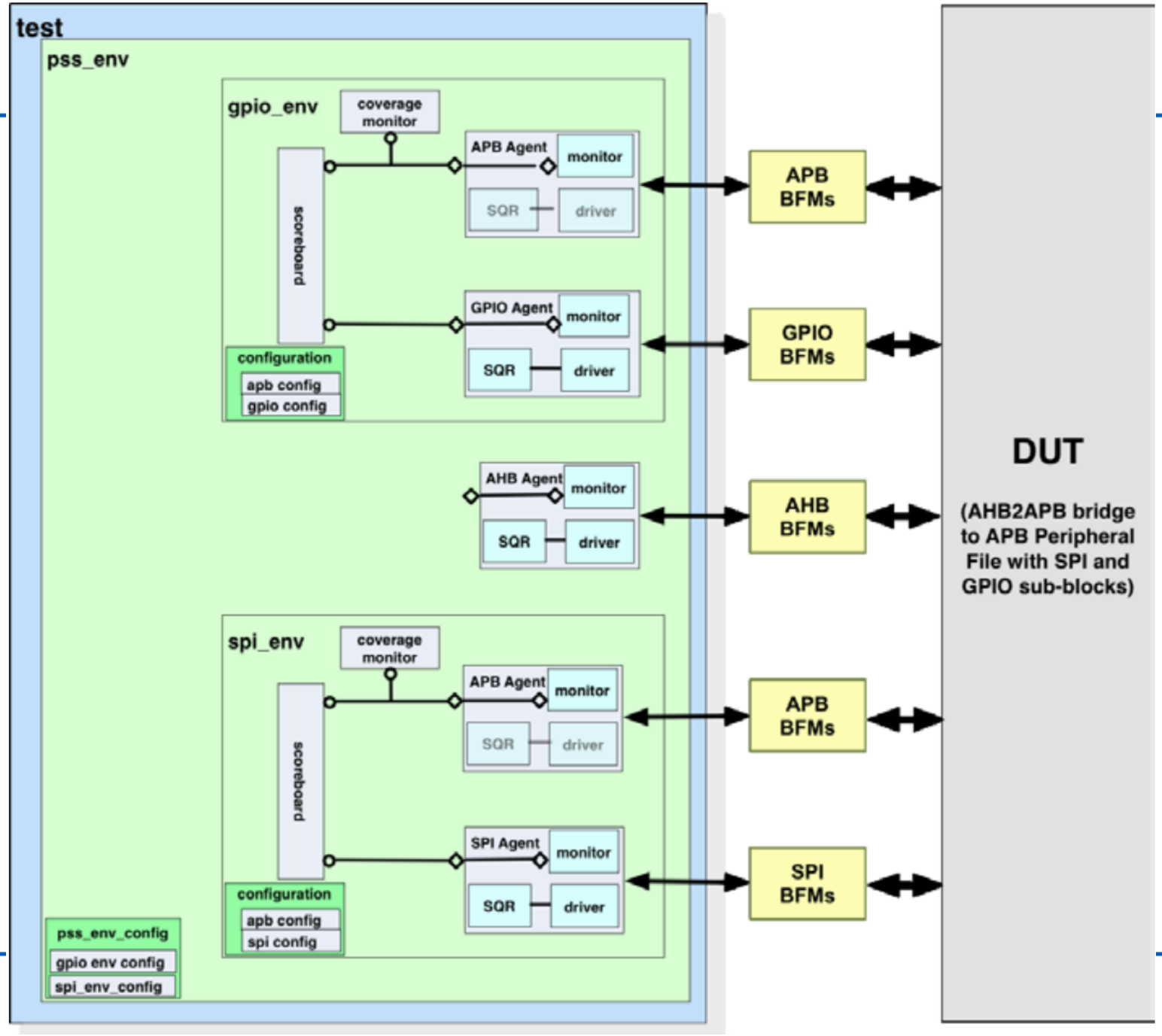
Configuração do Agent

- uvm_config_db
  - Modo de Operação (Ativo/Passivo)
  - Parâmetros do Protocolo
    - rand int data_bits; // 5,6,7,8 bits
    - rand bit parity_enable;
    - rand parity_type_e parity;
  - Comportamento do Driver
    - rand bit inject_errors;
    - rand real error_rate;
    - rand error_type_e error_type; // BIT_FLIP, FRAME, PARITY
    - rand driver_mode_e mode; // NORMAL, BURST, CONTINUOUS
  - Comportamento do Monitor
    - rand bit collect_coverage = 1;
    - rand bit enable_protocol_checks = 1;
    - rand bit_type_e monitored_bits; // ALL, ONLY_DATA, CONTROL_ONLY
    - rand sampling_method_e sampling; // POSEDGE, NEGEDGE, BOTH

Configuração do Environment

- uvm_config_db
  - Componentes do Ambiente
    - rand bit has_scoreboard = 1;
    - rand bit has_coverage = 1;
    - rand sync_method_e sync_method; // EVENT, SEMAPHORE, MAILBOX
  - Configurações de Sistema/Subsistema
    - rand system_mode_e system_mode; // STANDALONE, INTEGRATED, SOC
    - rand real system_clock_freq = 100e6; // 100 MHz
    - rand real peripheral_clock_freq = 50e6;
    - rand clock_ratio_e clock_ratio; // SYNC, ASYNC
  - Configurações de Verificação Global
    - rand real coverage_goal = 95.0; // %
    - rand bit [31:0] error_threshold = 10;
    - rand real test_timeout = 1.0e6;

Env_config


test
pss_env
gpio_env
coverage monitor
APB Agent
monitor
SQR
driver
scoreboard
GPIO Agent
monitor
SQR
driver
configuration
apb config
gpio config
AHB Agent
monitor
SQR
driver
spi_env
coverage monitor
APB Agent
monitor
SQR
driver
scoreboard
SPI Agent
monitor
SQR
driver
configuration
apb config
spi config
pss_env_config
gpio env config
spi_env_config
APB BFMs
GPIO BFMs
AHB BFMs
APB BFMs
SPI BFMs
DUT
(AHB2APB bridge to APB Peripheral File with SPI and GPIO sub-blocks)

Configuração do teste

- uvm_config_db (Via Plusargs)
  - xrun +UVM_TESTNAME=test_uart_high_speed +uvm_set_config_int=*,target_baud_rate,115200
  - // Arquivo: test_uart_high_speed.sv (UVC com vector set)

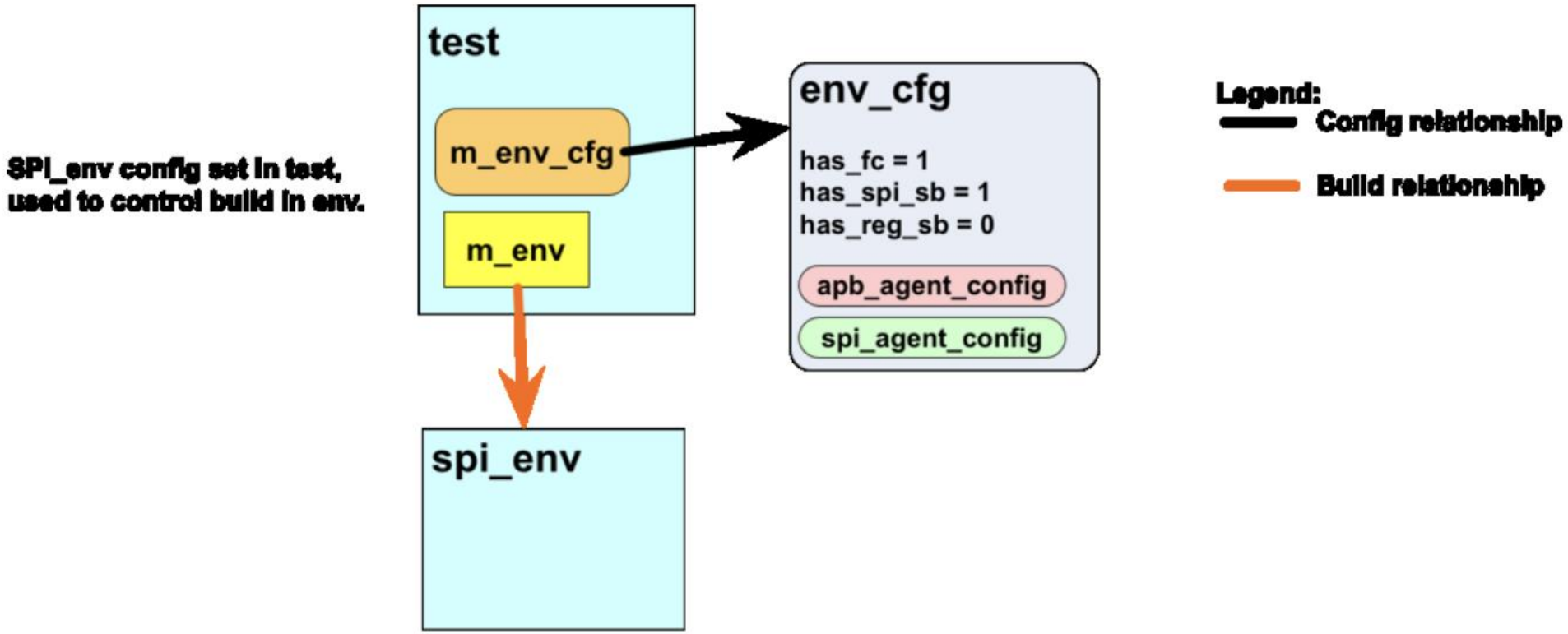
```
class test_uart_high_speed extends uvm_test;
  ...
  uart_config test_cfg;
  function void build_phase(uvm_phase phase);
    int baud_rate;
    test_cfg = uart_config::type_id::create("test_cfg", this);
    // 3. RECUPERE o valor configurado pela linha de comando
    if (uvm_config_db#(int)::get(null, "", "target_baud_rate", baud_rate)) begin
      test_cfg.target_baud_rate = baud_rate;
      `uvm_info(get_full_name(),  $sformatf("Configured high speed: %0d", baud_rate), UVM_LOW)
    end else begin ...
    // 4. COLOQUE o objeto de configuração no db, para ser recuperado pelo ENV/AGENT.
    // O path '*' garante que qualquer componente pode pegá-lo.
    uvm_config_db#(uart_config)::set(this, "*", "test_config_obj", test_cfg)
```

Precedência

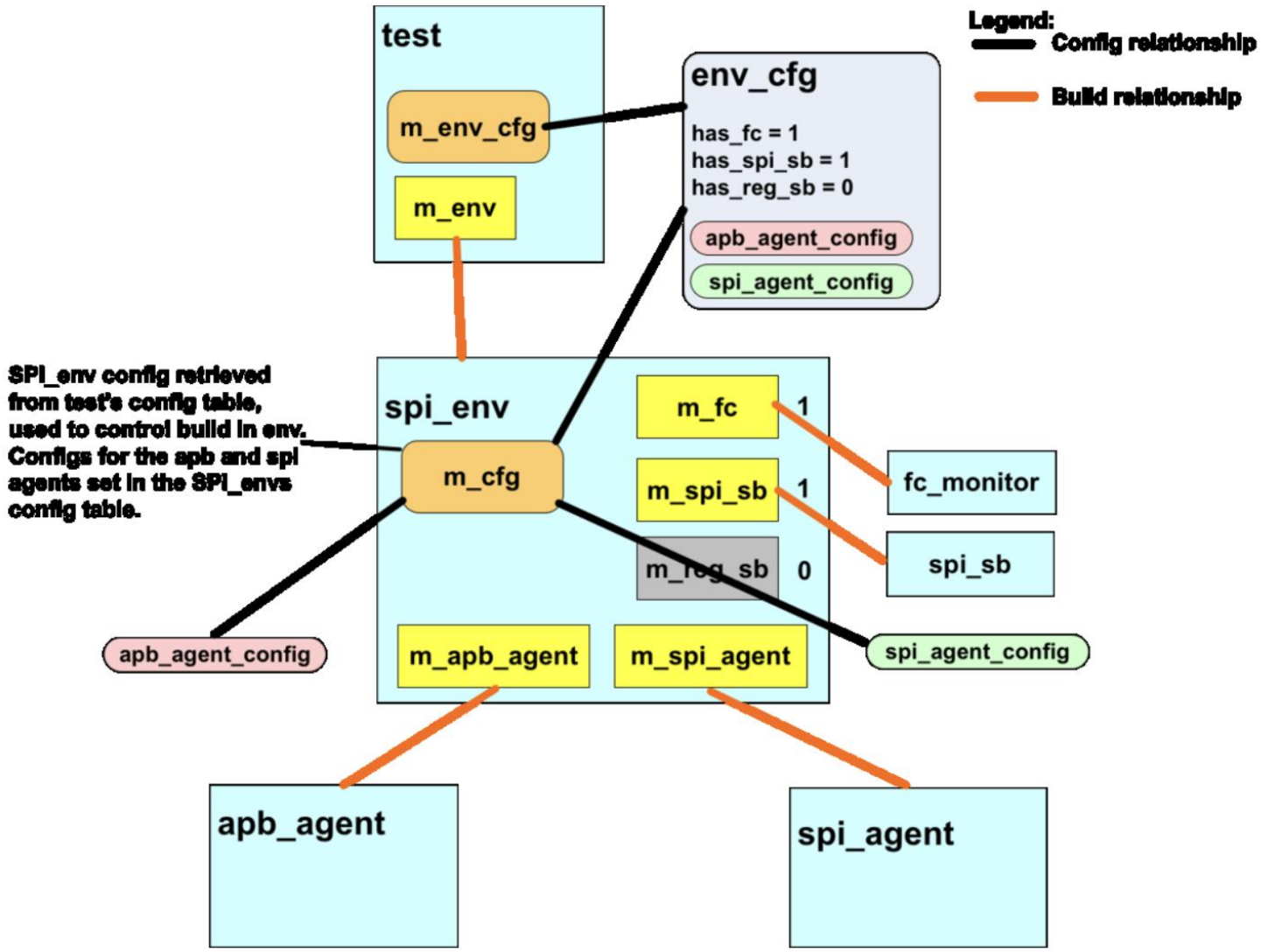
- uvm_config_db
  - Durante a fase de construção (build phase), uma chamada set() em um contexto mais alto na hierarquia de componentes tem precedência sobre uma chamada set() ocorrendo mais abaixo na hierarquia.
  - Em caso de contextos idênticos ou após a fase de construção, a última chamada set() tem precedência sobre a anterior.

UVM Testbench

- teste



UVM Testbench



Parametrização

- No SystemVerilog, mudar um parâmetro em uma interface gera um tipo de dado totalmente novo.

- Consistência rígida entre RTL e TB
  - O Virtual Interface, Driver, Monitor e o uvm_config_db devem usar exatamente os mesmos parâmetros do RTL.

  - Parametrizar classe por classe polui o código, dificulta a manutenção e gera erros de incompatibilidade de tipos.

Parametrização

- Centralização Única
  - Concentra todos os parâmetros do projeto em um único pacote de testbench (tb_params_pkg).
  - Uso de typedefs para cria apelidos de tipo para as Virtual Interfaces e agentes UVM parametrizados.
- Consistência Automática
  - Alinhar as chamadas de uvm_config_db::set() e get() em toda a hierarquia UVM sem código duplicado.

```
package tb_params_pkg;
    parameter int DATA_WIDTH = 32;
    parameter int ADDR_WIDTH = 16;
    typedef virtual my_if #(DATA_WIDTH, ADDR_WIDTH) vif_t;
endpackage
```

Parametrização

- Parametrização Mista
  - Parâmetros gerenciam as interfaces, enquanto as sequence_items utilizam a largura máxima suportada (ex: 32 bits).
- Sequências Limpas
  - As sequences do UVM não precisam de parâmetros, simplificando a escrita e reuso dos testes.
- Tratamento no VIP
  - O Driver e o Monitor mascaram os bits, transmitindo apenas os dados ativos na interface real.

Parametrização

```
class my_driver extends uvm_driver #(my_seq_item);
  virtual tb_params_pkg::vif_t vif;

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(tb_params_pkg::vif_t)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "Virtual interface not set")
  endfunction
endclass
```

Parametrização

- package my_uvc_pkg;
- import tb_params_pkg::*;
-
- `include "my_driver.sv"
- `include "my_monitor.sv"
- `include "my_agent.sv"
- ...
- endpackage

Exercício

- Criar ambiente UVM para um design genérico "DUT", com o agent config contendo a virtual interface, agente ativo/passivo, scoreboard on/off, coverage colector on/off e criação dos pacotes necessários.

Página em branco.